AIMERZ.ai
Aim.Act.Achieve

NOTES

Interfaces in Java - II

Implementing an Interface
Necessity of Interface with Use Cases
Interface Vs Abstraction

Implementing an Interface

When a class uses (implements) an interface, it **must** provide concrete implementations for all the abstract methods declared in the interface.

Example 1:
Here's how different vehicles can implement the `Movable` interface to define their specific movements.

```Java
class Truck implements Movable {
    public void move() {
        System.out.println("The truck moves on highways.");
    }
}

class Boat implements Movable {
    public void move() {
        System.out.println("The boat sails on water.");
    }
}
```

Example2: Implementing a Payment Interface

```Java
interface Payment {
    void pay(double amount);
}
class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using Credit Card.");
    }
}
class PayPalPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount + " using PayPal.");
    }
}
public class Main {
    public static void main(String[] args) {
        Payment payment = new CreditCardPayment();
```

```
        payment.pay(100.0);  // Output: Paid 100.0 using Credit Card.

        payment = new PayPalPayment();
        payment.pay(150.0);  // Output: Paid 150.0 using PayPal.
    }
}
```

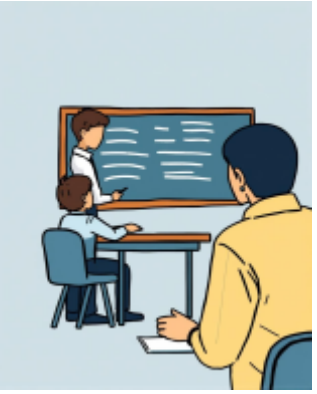
Interface Usage:
Interfaces allow us to define a family of similar classes (like all vehicles) that each implement a specific action in their own way. This approach is helpful when creating software that works with multiple, diverse objects that share common actions or behaviors.

Default and Static Methods in an Interface

Java 8 introduced **default** and **static methods** in interfaces. These methods are helpful because they allow interfaces to have some actual code (method bodies) instead of only abstract methods.

Default Methods: These methods provide default behavior in the interface. Classes implementing the interface can use the default implementation or override it if they need a custom behavior.

Real World Analogy: Teacher
Think of it like a teacher who gives general instructions to students. Every student can follow these instructions, but if a student has a unique way to complete the task, they can modify it.



Example:

```
Java
interface Printer {
    void print();

    // Default method
```

```
    default void scan() {
        System.out.println("Scanning is available.");
    }
}

class OfficePrinter implements Printer {
    public void print() {
        System.out.println("Printing a document.");
    }

    // Override scan
    public void scan() {
        System.out.println("Office printer scans with high resolution.");
    }
}

class HomePrinter implements Printer {
    public void print() {
        System.out.println("Printing at home.");
    }
    // Inherits default scan() method
}
```

Static Methods:
These are utility methods in interfaces that can be called without creating an instance of the implementing class. They are usually helper methods.

Example

```
Java
interface MathOperations {
    static int add(int a, int b) {
        return a + b;
    }
}
```

Necessity of Interface with Use Cases

Interfaces are necessary when different classes need to share a set of behaviors, especially when:

Multiple Inheritance Limitations: Java does not support multiple inheritance (a class inheriting from more than one superclass). Interfaces allow a class to inherit behaviors from multiple sources.

Standardization: Interfaces enforce a contract, ensuring that classes implementing the interface will have certain methods, helping maintain consistency in code.

Loose Coupling: Interfaces allow us to design systems with loose coupling. They help in separating the "what" from the "how", making the system more maintainable and extensible.

Example Use Case: Payment System

```Java
interface Payment {
    void makePayment(double amount);
}
class StripePayment implements Payment {
    public void makePayment(double amount) {
        System.out.println("Payment of $" + amount + " made via Stripe.");
    }
}
class PayPalPayment implements Payment {
    public void makePayment(double amount) {
        System.out.println("Payment of $" + amount + " made via PayPal.");
    }
}
class PaymentProcessor {
    public void processPayment(Payment payment, double amount) {
        payment.makePayment(amount);
    }
}
public class Main {
    public static void main(String[] args) {
        Payment stripe = new StripePayment();
        Payment payPal = new PayPalPayment();

        PaymentProcessor processor = new PaymentProcessor();
        processor.processPayment(stripe, 200.0); // Output: Payment of 200.0
made via Stripe.
        processor.processPayment(payPal, 300.0); // Output: Payment of 300.0
made via PayPal.
    }

}
```

Interface vs. Abstract Class

An interface is entirely abstract, meaning all methods (before Java 8) had to be abstract. An abstract class can have a mix of abstract and non-abstract methods.

Feature	Interface	Abstract Class
Methods	Abstract (or default, static)	Abstract and concrete methods
Fields	Only final and static	Can have any type of field
Inheritance	Multiple	Single inheritance

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan